Caso práctico

Materiales formativos de FP Online propiedad del Ministerio de Educación, Cultura y Deporte.

1.- Características de las bases de datos objeto-relacionales.

Caso práctico

Ana ha sabido que **Juan** ha realizado el curso de formación de bases de datos objeto-relacionales, y éste se ha ofrecido a compartir los conocimientos que ha adquirido. Lo primero va a ser es que conozca las características que tienen estos tipos de bases de datos, para que compruebe las diferencias que tienen respecto a las bases de datos relaciones que ha usado hasta ahora. Usarán la base de datos de Oracle que han utilizado anteriormente, pero ahora van a darle el enfoque de este nuevo tipo de bases de datos.

Las **bases de datos objeto-relacionales** que vas a conocer en esta unidad son las referidas a aquellas que han evolucionado desde el modelo relacional tradicional a un modelo híbrido que utiliza además la tecnología orientada a objetos. Las **clases, objetos,** y **herencia** son directamente soportados en los esquemas de la base de datos y el lenguaje de consulta y manipulación de datos. Además da soporte a una extensión del modelo de datos con la creación personalizada de **tipos de datos y métodos**.

La base de datos de Oracle implementa el modelo orientado a objetos como una **extensión del modelo relacional**, siguiendo soportando la funcionalidad estándar de las bases de datos relacionales.

El modelo objeto-relacional ofrece las **ventajas** de las técnicas orientadas a objetos en cuanto a mejorar la reutilización y el uso intuitivo de los objetos, a la vez que se mantiene la alta capacidad de concurrencia y el rendimiento de las bases de datos relacionales.

Los tipos de objetos que vas a conocer, así como las características orientadas a objeto, te van a proporcionar un mecanismo para **organizar los datos y acceder a ellos a alto nivel**. Por debajo de la capa de objetos, los datos seguirán estando almacenados en columnas y tablas, pero vas a poder trabajar con ellos de manera más parecida a las entidades que puedes encontrar en la vida real, dando así más significado a los datos. En vez de pensar en términos de columnas y tablas cuando realices consultas a la base de datos, simplemente deberás seleccionar entidades que habrás creado, por ejemplo clientes o pedidos.

Podrás utilizar las características orientadas a objetos que vas a aprender en esta unidad a la vez que puedes seguir trabajando con los datos relacionalmente, o bien puedes usar de manera más exclusiva las técnicas orientadas a objetos.

En general, el modelo orientado a objetos es **similar al que puedes encontrar en lenguajes** como C++ y Java. La **reutilización** de objetos permite desarrollar aplicaciones de bases de datos más rápidamente y de manera más eficiente. Al ofrecer la base de datos de Oracle soporte nativo para los tipos de objetos, permite a los desarrolladores de aplicaciones con lenguajes orientados a objetos, acceder directamente a las **mismas estructuras de datos** creadas en la base de datos.

La programación orientada a objetos está especialmente enfocada a la construcción de componentes reutilizables y aplicaciones complejas. En **PL/SQL**, la programación orientada a objetos está basada en **tipos de objetos**. Estos tipos te permiten modelar objetos de la vida real, separar los detalles de los interfaces de usuarios de la implementación, y almacenar los datos orientados a objetos de forma permanente en una base de datos. Encontraras especialmente útil los tipos de objetos cuando realizas programas interconectados con Java u otros lenguajes de programación orientados a objetos.

Las tablas de bases de datos relacionales sólo contienen datos. En cambio, los objetos pueden incluir la posibilidad de realizar determinadas **acciones sobre los datos**. Por ejemplo, un objeto "Compra" puede incluir un método para calcular el importe de todos los elementos comprados. O un objeto "Cliente" puede tener métodos que permitan obtener su historial de compras. De esta manera, una aplicación tan sólo debe realizar una llamada a dichos métodos para obtener esa información.

Para saber más

Si deseas conocer más en detalle la definición de las bases de datos objeto-relacionales puedes visitar la web de wikipedia:

Definición de base de datos objeto-relacional.

2.- Tipos de datos objeto.

Caso práctico

Ada Lovelace (cabeceira) debate con la ayuda conceptos básicos en los datos de tipo objeto.

Los **tipos de datos objeto** son un tipo de dato complejo definido por el usuario. Recordando, una **estructura de datos** es el modo **funcional, y posiblemente eficiente**, para manipular datos. Como ya sabemos, hay variables de un solo elemento (los tipos de datos simples: **cadenas, enteros, decimales, fecha**, etc.) pueden almacenar a la vez varios datos relacionados (perfiles, como ya sabemos, almacenar varios datos en una misma variable simple: todos del mismo tipo o los tipos de datos objeto más potentes: almacenan datos de distintos tipo, y posiblemente variables que son listas) de forma que podamos acceder a ellos de forma individual o como bloque(s), y de modo, tanto uno, como el otro, **eficiente**.

Un ejemplo de tipo objeto podría ser el de una persona, que podría tener **propiedades** como: nombre, apellidos, edad, dirección, teléfono, etc. y **métodos** como: hablar, andar, comer, etc.

Reflexiona

Piensa en un objeto y piensa algunas de sus características y métodos que se puedan modelar similar a

Los objetos son **tipos de datos** que se utilizan para modelar el mundo real. **Atributos** (pueden contemplar de uno o más propiedades) y sus funciones y **métodos** (funciones de tipo de dato objeto) de forma **relacionada**.

Por ejemplo, un objeto de la **entidad** de tipo de dato objeto **persona** podría tener **atributos** (propiedades) que se **definen** cuando se **crea** el objeto: **nombre, fecha de nacimiento**, etc. Y también podría tener **métodos** que le den funcionalidad, como **calcular edad**, que podría calcular la edad de la persona a partir de su fecha de nacimiento.

Aunque las definiciones de objetos, en este contexto, suelen ofrecer programas enteros, los programas también se **manipulan fácilmente unitariamente**. Esto significa que podemos trabajar con un único objeto, o incluso con varios diferentes. Esto nos permite, por ejemplo, ordenar una colección de objetos, o incluso buscar en ellos, de forma sencilla.

Finalmente, como hemos dicho en el mundo de la orientación a objetos, los programas están **compuestos principalmente** de objetos, que se comunican entre sí. Hablamos de **mensajes** cuando **enviamos** un objeto a otro, para que realice una de sus funciones. Por ejemplo, si una persona tiene un método **calcular edad**, podríamos enviarle un mensaje para que lo realice.

3.- Definición de tipos de objeto.

Caso práctico

Ya sabemos utilizar un tipo de base de datos orientada a objetos y ya conocíamos los tipos de datos objeto con los que podíamos definir columnas de tablas y emplear, con la ayuda de **SQL**, todas las ventajas de la orientación a objetos que en una fecha previa, en el material que, en el Power, vas hacer de tablas orientadas a objetos.

Pero claro, se plantea crear el tipo de objeto Usuario, pero necesita conocer cómo se representan los tipos de datos objeto en Oracle.

Ya sabemos lo que es un objeto y también sabemos que **una clase** es una especificación de un objeto. Una clase contiene **atributos** y métodos. Los atributos son como las características o estados y pueden pertenecer a dos clases: de un **tipo** que se implementa en Oracle y de otro tipo, **colección**.

Vamos a ver primero cómo se crea un objeto de un tipo. Un objeto es una instancia de una clase. Los atributos de un objeto se definen en la especificación de la clase y los métodos se implementan en el cuerpo de la clase.

En la especificación no se dice de qué tipo son los atributos de entrada, es decir, **el número de parámetros y el nombre de cada uno de ellos**. Pero sí se dice si son de entrada, de salida o de entrada y salida. Además, debemos declarar todos los métodos que no vayamos a usar. Debes tener en cuenta también que el cuerpo de la clase no podemos usar todos los métodos que no hayamos declarado en la especificación. Por último, debemos declarar el constructor del tipo de objeto y podemos, si lo deseamos, sobrecargarlo, es decir, definir tantos tipos de objeto.

Por tanto, un tipo de objeto contiene una especificación, donde se declaran atributos y métodos, y el cuerpo de la clase, donde se definen los métodos. Los atributos son variables de instancia, y el cuerpo de la clase contiene las implementaciones de los métodos declarados en la especificación. Por último, el constructor es una función que se utiliza para crear un objeto de un tipo de objeto.

Para definir un objeto en Oracle debes utilizar la sentencia **CREATE TYPE** que tiene un gran abanico de usos:

```
CREATE TYPE nombre_tipo AS OBJECT (
  declaración_atributos
  declaración_métodos
);
```

Vamos a continuar, igual que cuando trabajamos paso a paso los objetos Usuario de la unidad de trabajo anterior, con el ejemplo que hicimos de una agenda de contactos.

En vez de usar **el nombre del tipo de objeto ya utilizado anteriormente** para el tipo de objeto en Oracle, vamos a usar el nombre Empleado, ya que el concepto que se quiere almacenar es el de un empleado. Luego, **ya** se encargará de la declaración del tipo de objeto.

```
CREATE OR REPLACE TYPE nombre_tipo AS OBJECT
```

Por ejemplo, queremos el tipo de objeto Usuario, **inmediatamente** después de poner "tipo objeto" y la palabra AS, que nos va a indicar que vamos a declarar un tipo de objeto, vamos a poner los atributos que queremos que tenga dicho objeto. Y a continuación, entre paréntesis, los métodos que queremos que tenga dicho objeto.

```
CREATE OR REPLACE TYPE Usuario AS OBJECT (
  declaración_atributos
  declaración_métodos
);
```

Por último, indicamos la palabra **END** para que todo quede bien cerrado.

```
END TYPE nombre_tipo;
```

Vamos a definir, tipo dato por tipo dato, los atributos que deseamos almacenar. Por ejemplo, queremos tipo de objeto Usuario, almacenar nombre:

```
END TYPE Usuario;
```

3.1.- Declaración de atributos.

La declaración de los atributos la puedes realizar de forma muy similar a declaración de las variables, es decir, utilizando un nombre y un tipo de dato. Dicho nombre debe ser único dentro del tipo de objeto, aunque puede ser reutilizado en otros tipos de objeto. El tipo de dato que puede almacenar un determinado atributo puede ser **cualquiera de los tipos de Oracle excepto** los siguientes:

- LONG y LONG RAW.
- ROWID y UROWID.
- Los tipos específicos PL/SQL BINARY_INTEGER (y sus subtipos), BOOLEAN, PLS_INTEGER, RECORD, REF CURSOR, %TYPE, y %ROWTYPE.
- Los tipos definidos dentro de un paquete PL/SQL.

Debes tener en cuenta que no puedes inicializar los atributos usando el operador de asignación, ni la cláusula DEFAULT, ni asignar la restricción NOT NULL.

El **tipo de dato de un atributo puede ser otro tipo de objeto**, por lo que la estructura de datos puede ser tan complicada como sea necesario.

```
CREATE OR REPLACE TYPE Usuario AS OBJECT (
        login VARCHAR2(10),
        nombre VARCHAR2(30),
        f_ingreso DATE,
        credito NUMBER
);
/
```

Después de haber sido creado el tipo de objeto, se pueden **modificar sus atributos** utilizando la sentencia ALTER TYPE. Si se desean añadir nuevos atributos se añadirá la cláusula ADD ATTRIBUTE seguida de la lista de nuevos atributos con sus correspondientes tipos de dato. Utilizando MODIFY ATTRIBUTE se podrán modificar los atributos existentes, y para eliminar atributos se dispone de manera similar de DROP ATTRIBUTE.

Aquí tienes varios ejemplos de modificación del tipo de objeto Usuario creado anteriormente:

```
ALTER TYPE Usuario DROP ATTRIBUTE f_ingreso;

ALTER TYPE Usuario ADD ATTRIBUTE (apellidos VARCHAR2(40), localidad VARCHAR2(50));

ALTER TYPE Usuario
        ADD ATTRIBUTE cp VARCHAR2(5),
        MODIFY ATTRIBUTE nombre VARCHAR2(35);
```

Para saber más

En la web de Oracle puedes encontrar la sintaxis completa de la instrucción CREATE TYPE. Ahí podrás conocer que hay más posibilidades de uso:

Sintaxis completa de la instrucción CREATE TYPE. (En inglés)

En la web de Oracle puedes encontrar la sintaxis completa de la instrucción ALTER TYPE. Ahí podrás conocer que hay más posibilidades de uso:

Sintaxis completa de la instrucción ALTER TYPE. (En inglés)

3.2.- Definición de métodos.

Un **método es un subprograma** que declaras en la especificación de un tipo de objeto usando las palabras clave MEMBER o STATIC. Debes tener en cuenta que el nombre de un determinado método no puede ser el mismo nombre que el tipo de objeto ni el de ninguno de sus atributos. Como se verá más adelante, se pueden crear métodos con el mismo nombre que el tipo de objeto, pero dichos métodos tendrán una función especial.

Al igual que los subprogramas empaquetados, los métodos tienen **dos partes: una especificación y un cuerpo**.

En la **especificación** o declaración se debe encontrar el nombre del método, una lista opcional de parámetros, y, en el caso de las funciones, un tipo de dato de retorno. Por ejemplo, observa la especificación del método incrementoCredito que se encuentra detrás de las declaraciones de los atributos:

```
CREATE OR REPLACE TYPE Usuario AS OBJECT (
        login VARCHAR2(10),
        nombre VARCHAR2(30),
        f_ingreso DATE,
        credito NUMBER,
        MEMBER PROCEDURE incrementoCredito(inc NUMBER)
);
/
```

En el **cuerpo** debes indicar el código que se debe ejecutar para realizar una determinada tarea cuando el método es invocado. En el siguiente ejemplo se desarrolla el cuerpo del método que se ha declarado antes:

```
CREATE OR REPLACE TYPE BODY Usuario AS
        MEMBER PROCEDURE incrementoCredito(inc NUMBER) IS
                BEGIN
                        credito := credito + inc;
                END incrementoCredito;
END;
/
```

Por cada especificación de método que se indique en el bloque de especificación del tipo de objeto, debe existir su correspondiente cuerpo del método, o bien, el método debe declararse como NOT INSTANTIABLE, para indicar que el cuerpo del método se encontrará en un subtipo de ese tipo de objeto. Además, debes tener en cuenta que las cabeceras de los métodos deben coincidir exactamente en la especificación y en el cuerpo.

Al igual que los atributos, los parámetros formales se declaran con un nombre y un tipo de dato. Sin embargo, el tipo de dato de un parámetro no puede tener restricciones de tamaño. El tipo de datos puede ser cualquiera de los empleados por Oracle salvo los indicados anteriormente para los atributos. Las mismas restricciones se aplican para los tipos de retorno de las funciones.

El código fuente de los métodos no sólo puede escribirse en el lenguaje PL/SQL. También con otros lenguajes de programación como Java o C.

Puedes usar la sentencia ALTER TYPE para **añadir, modificar o eliminar métodos** de un tipo de objeto existente, de manera similar a la utilizada para modificar los atributos de un tipo de objeto.

3.3.- Parámetro SELF

La propiedad **s-parent** (que puedes utilizar con las variantes **SELFCHILD** y **SELFSIBLING**) se refiere a la clase **s-parent**. Debes poner la clase **s-parent** en el elemento padre. Veamos un ejemplo:

En la figura **3.3.1** tenemos dentro de un elemento padre con clase s-parent dos hijos: el hijo de tipo fracciones (**SELFCHILD**) y el hijo de tipo decimales (**SELFSIBLING**). Ambos tienen la misma clase **fracciones_y_decimales**, pero el hijo de tipo fracciones tiene además la clase **SELFCHILD**, y el hijo decimales tiene la clase **SELFSIBLING**.

En el caso hijo fracciones se ha añadido la clase **display-flex-column-50-100-100**, haciendo que este fracción1 ocupe el 50% de la pantalla en ordenadores, el 100% en tablets y el 100% en móviles.

```
Código: fracciones_y_decimales.html | CSS: SELF.CSS

<div class="s-parent">
    <!-- el primer elemento, clase selfchild, tiene colocarse en el mismo eje Y que los siguientes pues se
    <div class="fracciones_y_decimales display-flex-column-50-100-100 selfchild">
        <h3>Fracción: 1</h3>
    </div>
```


Autoevaluación

¿Cuál de las afirmaciones sobre el parámetro SELF es correcta?

- a) Falsa
- b) Verdadera
- c) Depende del caso
- d) Indefinida

3.4.- Sobrecarga

En gran parte el poder de la programación orientada a objetos, nos referimos puntos en común de programas, es decir, poder **utilizar el mismo nombre para varios métodos diferentes** siempre y cuando sea una de las siguientes formas que diferencien por cantidad o tipo de datos.

Cuando nos hace una declaración en el método, nos vimos que los parámetros formales de los métodos son una lista separada, y es aquí donde vamos a ver como varios métodos de un polimorfismo.

Ya vimos cuando que no es posible la sobrecarga de dos métodos, cuyos parámetros formales se diferencian únicamente en su nombre, pero ahora bien para que diferenciar correctamente en el cuerpo de sentencia.

Debes tener siempre en cuenta, ya que se diferencian en el número de parámetros.

```
double CalcArea(double radius);
double CalcArea(double radius, double height);
```

Pero existe otra clase y nuevos también son el que se utilizan los nuevos tipos de parámetros formales aunque sea diferenciar los nombres de los parámetros.

```
double CalcArea(double radius);
double CalcArea(int radius);
```

Autoevaluación

¿Reconoce/declara siguientes diferenciación de métodos?

```
double CalcTotal(double[] array);
double CalcTotal(int[] array);
```

Verdadero
Falso

3.5.- Métodos Constructores

Cada tipo de objeto tiene un método constructor, que es una **función con un nombre idéntico que el tipo de objeto** y que se encarga de **inicializar los atributos y ejecutar una nueva instancia** de uno de los objetos.

Puede haber un **método constructor por defecto** que tiene tipo de objeto sin parámetros, o puede personalizarse tomando parámetros y más. Un tipo puede tener varios métodos constructores.

También puedes definir **los propios métodos constructores**, especificando el valor deseado mediante parámetros por el usuario, o fijos, definidos en el cuerpo método del constructor personalizado en el que se pueden hacer una verificación de que todos los atributos que se van a asignar a los atributos son correctos (por ejemplo, que no tengan un valor demasiado numeroso).

En ese caso, siempre el método constructor por defecto, debes utilizar la palabra reservada NEW seguida del nombre del tipo de objeto en el que pueda encontrarlo (recuerda que un método constructor personalizado es un método cuando una función de una instancia, que el tipo de objeto), la cual puede tener diferentes formas de parámetros que usen. Asumiendo de la manera habitual, FM offers, debes indicar que un valor del último de la función, así el tipo objeto utilizando la cláusula **AUTO DECLARE** de **AMBOS!**

Puedes crear **varios métodos constructores** siguiendo las restricciones indicadas para la sobrecarga de métodos.

Por el siguiente ejemplo puedes ver la declaración y el cuerpo de un método constructor personalizado llamado **CreaVector**, que se encarga de **inicializar los atributos** de una clase objeto, con un método constructor y sobrecarga de métodos.

```
CREATE OR REPLACE TYPE vector(x NUMBER, y NUMBER)
    AS OBJECT (x1 NUMBER,
               y1 NUMBER,
CREATE OR REPLACE TYPE BODY vector AS
    CONSTRUCTOR FUNCTION vector(x NUMBER, y NUMBER)
        RETURN SELF AS RESULT
    IS
        BEGIN
            IF (x=NULL or y= NULL)
                THEN x:=0; y:=0;
            END IF;
            SELF.x1:=x;
            SELF.y1:=y;
            RETURN;
        END;
END;
```

4.- Utilización de objetos.

Caso práctico

Una vez que dejamos las nomenclaturas conceptos personal, hardware y tipo de datos atrás, vamos a conocer la forma en la cual interactúan para lograr el objetivo de una tarea dada.

Por ello seguimos nuestros amigos con Chris y Linda **Herman Urbitbiz** que continúa diciendo en alguna, ahora con **Alberto Urbitbiz** y **Luis Urbitbiz**, acerca de **la orientación** de ellos y diferencia así: cualquier momento, y **hacer a las máquinas** que ahorre el tipo de objeto utilizado.

4.1.- Declaración de objetos.

Una vez que el tipo de objeto ha sido definido, debe usarse para declarar **variables de objeto**, de una forma un poco más formal de lo habitual. Una clase de objeto incorporará diferentes tipos de datos (variables), atributos, relacionados con esa misma, predefinidos de forma y modelados de una forma tal, de que el resultado pueda ser utilizado tipos de datos fundamentales como Void Boolean, Int **Unidad**.

Por ejemplo, una variable de este tipo Unidad, se puede definir como un objeto de clase Unidad, utilizar todos los métodos y propiedades.

```
var Unidad tmp;
```

En la declaración hay ocupar **(constituidores y función)**, incluyendo los métodos de inserción de objetos de datos, se puede utilizar un tipo de datos objeto definido para todos los que debe **palabra clave pertinente un objeto de**. Within tipo a la llamada. Por ejemplo (ejemplos en un procedimiento de la que esta debe pasar como parámetro) un objeto del tipo Unidad.

```
DISCIPLINE: setUnidad(tmp: Unidad);
```

y la referencia a esta entidad, se mantenga utilizar de nuevo y pertenecer con objeto, como en la que podemos llevar a la variable bombardeadaInternamente

```
setUnidad(Unidad);
```

Por ejemplo para el proceso de la **función objeto**:

```
CRIMTRIM: getUnidad(Unidad tmp): BOOLEAN; BOOLEAN;Unidad;
```

Los objetos se crean mientras se ejecuten más código como instrucciones del tipo de código, y realizará otro para contener valores diferentes en sus atributos.

En **métodos de clase objeto** que las clases objeto realizadas por la Web, y se dice de un bloque o subprograma encapsulado en variable, escritos en ellos bloque o subprograma, y se destruye, para **almacenamiento** cuando se sale de ellos. En un ejemplo, los objetos en la base destruye en el momento de hacer referencia a ejemplo y llegó a de usado cuando se finaliza la pueden ser no hace de datos.

Autoevaluación

Asumiendo que hemos declarado el tipo de objeto Factura, ¿Cuál de las siguientes declaraciones de variables para guardar un objeto de esa fuente correcta?

- Factura factura;
- factura2 = Factura;
- Factura Factura;

4.2.- Inicialización de objetos.

Para **crear o instanciar un objeto** de un determinado tipo de objeto, debes hacer una **llamada a su método constructor**. Esto lo puedes realizar empleando la **instrucción** NEW seguido del nombre del tipo de objeto como una llamada a una función en la que se indican como parámetros los valores que se desean asignar a los atributos inicialmente. En una asignación también puedes optar por hacer eso mismo omitiendo la palabra NEW.

El **orden de los parámetros** debe coincidir con el orden en el que están declarados los atributos, así como los tipos de datos. El formato sería como el siguiente:

```
variable_objeto := NEW Nombre_Tipo_Objeto (valor_atributo1, valor_atributo2, ...);
```

Por ejemplo, en el caso del tipo de objeto Usuario:

```
u1 := NEW Usuario('luitom64', 'LUIS ', 'TOMAS BRUÑA', '24/10/07', 100);
```

En ese momento se **inicializa el objeto**. Hasta que no se inicializa el objeto llamando a su constructor, el objeto tiene el valor NULL.

Es habitual inicializar los objetos en su declaración.

```
u1 Usuario := NEW Usuario('luitom64', 'LUIS ', 'TOMAS BRUÑA', '24/10/07', 100);
```

La llamada al método constructor se puede realizar en cualquier lugar en el que se puede hacer una llamada a una función de la forma habitual. Por ejemplo, la llamada al método constructor puede ser utilizada **como parte de una expresión**.

Los **valores de los parámetros** que se pasan al constructor cuando se hace la llamada, son asignados a los atributos del objeto que está siendo creado. Si la llamada es al método constructor que incorpora Oracle por defecto, debes indicar un parámetro para cada atributo, en el mismo orden en que están declarados los atributos. Ten en cuenta que los atributos, en contra de lo que ocurre con variables y constantes, no pueden tener valores por defecto asignados en su declaración. Por tanto, los valores que se desee que tengan inicialmente los atributos de un objeto instanciado deben indicarse como parámetros en la llamada al método constructor.

Existe la posibilidad de **utilizar los nombres de los parámetros formales** en la llamada al método constructor, en lugar de utilizar el modelo posicional de los parámetros. De esta manera no es obligatorio respetar el orden en el que se encuentran los parámetros reales respecto a los parámetros formales, que como se ha comentado antes coincide con el orden de los atributos.

```
DECLARE
u1 Usuario;
BEGIN
u1 := NEW Usuario('user1', -10);
/* Se mostrará el crédito como cero, al intentar asignar un crédito negativo */
dbms_output.put_line(u1.credito);
END;
/
```

Autoevaluación

¿Cuál de las siguientes inicializaciones de objetos es correcta para el tipo de objeto Factura, suponiendo que dispone de los atributos número (INTEGER), nombre (VARCHAR2) e importe (NUMBER)?

- ○ factura1 := NEW Factura(3, 'Juan Álvarez', 30.50);
- ○ factura1 = Factura(3, 'Juan Álvarez', 30.50);
- ○ factura1 := NEW Factura('Juan Álvarez', 3, 30.50);
- ○ factura1 := NEW (3, 'Juan Álvarez', 30.50);

4.3.- Acceso a los atributos de objetos.

Para hacer referencia a un atributo de un objeto debes utilizar el **nombre de dicho atributo, utilizando el punto** para enlazar el valor que contiene ó bien para modificarlo, si dicho atributo lo **permite el objeto** bajo ciertos criterios o condiciones.


```
usuario_objeto.usuario_atributo;
```

En ejemplo, la variable edad del tipo int, definida puede utilizarse como parte de una asignación o como parámetro en la llamada a una función:

```
edad=edad + 1;
Mifuncion(edad);
```

La **modificación del valor** contenido en el atributo puede ser similar a la siguiente:

```
usuario1.edad=usuario1.edad+1;
```

Los nombres de los atributos pueden ser encadenados, lo que permite el acceso a atributos de tipos de objetos anidados. Por ejemplo, si un objeto alfa1 tiene un atributo del tipo de objeto usuario, se accederá al atributo del nombre del usuario así:

```
alfa1.usuario1.edad;
```

Para referirse a una parte concreta de un atributo de un objeto (por ejemplo **un carácter** de una cadena de texto), se indicará entre corchetes [ ] la posición del carácter, dato de esta forma [posición]. Así, del texto "Hola mundo" el carácter "H" ocupa la posición [0], el carácter "a" la posición [1], el carácter "l" la posición [2], el carácter "o" la posición [3], el carácter "m" la posición [5], el carácter "u" la posición [6], el carácter "n" la posición [7], el carácter "d" la posición [8] y el carácter "o" la posición [9].

Para acceder a un carácter concreto de un atributo de un objeto (por ejemplo, para la función anterior), se accede así (ejemplo: **Mifuncion(usuario1.nombre[0])**). En el caso anterior, para acceder a la letra "H" de "Hola mundo" del atributo nombre del objeto usuario1, se accedería así: **usuario1.nombre[0]**. En el caso anterior, para acceder a la letra "u" de "Hola mundo" del atributo nombre del objeto usuario1, se accedería así: **usuario1.nombre[6]**.

Autoevaluación

¿Cuál de las siguientes instrucciones se encarga de asignar un nuevo valor al atributo edad de un objeto llamado usuario?

- a) usuario.edad=edad + 1;
- b) usuario.edad=edad + 1;
- c) edad=edad + 1;

4.4.- Llamada a los métodos de los objetos.

Al igual que hemos hecho a la hora de acceder a los atributos de un objeto, también podemos acceder a los métodos de un objeto **utilizando un punto entre el nombre del objeto y el método**, como podemos observar en el siguiente ejemplo:

```
coche.frenar();
```

En el ejemplo anterior, se está accediendo al método **frenar** del objeto **coche** (definido en el ejemplo anterior) y se está ejecutando.

Los métodos, al contrario que los atributos, no tienen valor, sino que son bloques de instrucciones que se ejecutan cuando son llamados. Además, pueden recibir cero, uno o más valores (llamados parámetros) que se pasan a la hora de llamar al método.

Por ejemplo, si dispusiéramos de un objeto **perro** con un método de nombre **ladrar**, podríamos hacer que el objeto ladrase tres veces, pasando ese número como valor para indicarle a la función cuántas veces debe ladrar, de la siguiente manera:

```
perro.ladrar(3);
```

Los **métodos** son bloques de instrucciones utilizados cada vez que se llaman. Por lo tanto, no **contienen** ningún valor por defecto.

Sin embargo, sí es posible **definir** que un método acepte un tipo de objeto, en lugar de una de sus propiedades.

```
coche.frenar(pedal_freno);
```



```
coche.frenar(pedal_freno.valor);
```


Autoevaluación

¿Cuál de los siguientes métodos de acceder a **frenar** es el correcto para el objeto **factura**?

- coche.frenar(pedal_freno);
- coche.frenar(pedal_freno.valor);
- coche.frenar(pedal_freno);

4.5.- Herencia.

Hemos visto POO99, así como el concepto de clase y de objeto. Cuando hablé, creo recordar, puedes **definir subtipos** de los tipos de los objetos. Antes o después, **escribes un montón de atributos y métodos de tipos puntos**, para definirlos y puedes **definirlos subtipos/subclases**, o incluso, sobrescribir métodos de los tipos de.

Para entender esto, un tipo de objeto es **Serrucho**, de hecho hay uno que **cualquier ferretería tiene**, y además hay, para todos los usuarios de la clase. De cuales son los **métodos** de los tipos de **serrucho**? **cortar**, **dibujar**, y **no sé**, lo que se te ocurra. Pero, ¿y si quiero que la ferretería tenga **otro tipo de serrucho**? Pues tendría que definir un tipo de objeto nuevo, **serrucho2**, y además tendría que **copiarse todo el código** de la clase serrucho, y **sobrescribir** los métodos de objeto que deseamos cambiar, como dibujar, por ejemplo. Pero ¿y si hay algún error en la función de cortar? Pues tendría que corregirlo en dos sitios. Es un rollo.

En un mundo ideal, siempre se puede crear objetos plantilla para los tipos de objeto. Ejemplo: Serrucho es un tipo de objeto, pero **se puede crear objetos** Serrucho que tienen una forma de cortar la madera un poco distinta de otros tipos de objeto.

Por otro lado, siempre queremos poder usar todos los tipos de objeto. Pero ¿y si, por ejemplo, queremos usar todos los tipos de objeto que definimos, y además un tipo de objeto que definimos yo, pero que no existe? Pues, por ejemplo, queremos usar una variable que pueda ser de cualquier tipo de objeto, pudiendo almacenar una vez un número, luego una cadena para la lista de anteriores, o una persona. Pero, ¿cómo?

```
class TIPO_Serrucho_de_ARTest {
    cadena: ARRAYdeCAR;
    tipo2: clase_ARRAYdeCAR;
} ARTest;

class TIPO_Serrucho2 extends ARRAY Serrucho {
    // Sobrescribir métodos
    // Agregar más;
    // Cambiar todo
};

Serrucho

serrucho2

dibujar
    si ... ARRAY.dibujar(Serrucho.cadena[1] , Serrucho2 , nuevo1 , "Hecho.bien" , 999);
    dibujar_nuevo(Serrucho.cadena[1]);
ARRAY
```

Autoevaluación

Cuando un objeto que sea instanciado como clase hija de otro, ¿hereda los atributos y métodos del tipo de objeto padre?

- Verdadero
- Falso

5.- Métodos MAP y ORDER.

Caso práctico

Juan se ha dado cuenta de que va a tener un problema cuando necesite realizar comparaciones y órdenes entre objetos del mismo tipo.

Cuando tenga una serie de objetos de tipo Usuario, y necesite realizar una operación de ordenación entre ellos, ¿cómo debe hacerlo? Podría indicar que se realizaran las consultas de forma ordenada en función de alguno de los atributos del tipo de objeto. Ha estudiado el funcionamiento de los métodos MAP y ORDER, y ha comprobado que son los que le van a ofrecer un mecanismo sencillo para ello.

Las instancias de un tipo de objeto no tienen un orden predefinido. Si deseas establecer un orden en ellos, con el fin de **hacer una ordenación o una comparación, debes crear un método MAP**.

Por ejemplo, si haces una comparación entre dos objetos Usuario, y deseas saber si uno es mayor que otro, ¿en base a qué criterio se hace esa comparación? ¿Por el orden alfabético de apellidos y nombre? ¿Por la fecha de alta? ¿Por el crédito? Hay que establecer con un método MAP cuál va a ser el valor que se va a utilizar en las comparaciones.

Para crear un método MAP debes declarar un método que **retorne el valor que se va a utilizar para hacer las comparaciones**. El método que declares para ello debe empezar su declaración con la palabra MAP:

```
CREATE OR REPLACE TYPE Usuario AS OBJECT (
        login VARCHAR2(30),
        nombre VARCHAR2(30),
        apellidos VARCHAR2(40),
        f_ingreso DATE,
        credito NUMBER,
        MAP MEMBER FUNCTION ordenarUsuario RETURN VARCHAR2
);
/
```

En el cuerpo del método se debe retornar el valor que se utilizará para realizar las comparaciones entre las instancias del tipo de objeto. Por ejemplo, si se quiere establecer que las comparaciones entre objetos del tipo Usuario se realice considerando el orden alfabético habitual de apellidos y nombre:

```
CREATE OR REPLACE TYPE BODY Usuario AS
       MAP MEMBER FUNCTION ordenarUsuario RETURN VARCHAR2 IS
       BEGIN
              RETURN (apellidos || ' ' || nombre);
       END ordenarUsuario;
END;
/
```

El lenguaje PL/SQL utiliza los métodos MAP para evaluar expresiones lógicas que resultan valores booleanos como objeto1 > objeto2, y para realizar las comparaciones implícitas en las cláusulas DISTINCT, GROUP BY y ORDER BY.

A continuación puedes encontrar el código completo de un ejemplo de declaración y uso de un método MAP para establecer el orden en las comparaciones entre dos instancias de objetos del tipo Usuario. Como puedes comprobar, las comparaciones se realizan un función del orden alfabético de los apellidos seguidos del nombre.

Resumen textual alternativo

Cada tipo de objeto **sólo puede tener un método MAP declarado**, y sólo puede retornar alguno de los siguientes tipos: DATE, NUMBER, VARCHAR2, CHARACTER o REAL.

5.1.- Métodos ORDER.

De forma **similar al método** MAP, puedes declarar en cualquier tipo de objeto un método ORDER que te permitirá **establecer un orden** entre los objetos instanciados de dicho tipo.

Cada tipo de objeto **sólo puede contener un método** ORDER, el cual debe **retornar un valor numérico que permita establecer el orden entre los objetos.** Si deseas que un objeto sea menor que otro puedes retornar, por ejemplo, el valor -1. Si vas a determinar que sean iguales, devuelve 0, y si va a ser mayor, retorna 1. De esta manera, considerando el valor retornado por el método ORDER, se puede establecer si un objeto es menor, igual o mayor que otro en el momento de hacer una ordenación entre una serie de objetos del mismo tipo.

Para declarar qué método va a realizar esta operación, debes indicar la palabra ORDER delante de su declaración. Debes tener en cuenta que va a **retornar un valor numérico** (INTEGER), y que necesita que se indique un **parámetro que será del mismo tipo de objeto**. El objeto que se indique en ese parámetro será el que se compare con el objeto que utilice este método.

En el siguiente ejemplo, el sistema que se va a establecer para ordenar a los usuarios se realiza en función de los dígitos que se encuentran a partir de la posición 7 del atributo login.

```
CREATE OR REPLACE TYPE BODY Usuario AS
        ORDER MEMBER FUNCTION ordenUsuario(u Usuario) RETURN INTEGER IS
        BEGIN
                /* La función substr obtiene una subcadena desde la posición indicada hasta el final*/
                IF substr(SELF.login, 7) < substr(u.login, 7) THEN
                        RETURN -1;
                ELSIF substr(SELF.login, 7) > substr(u.login, 7) THEN
                        RETURN 1;
                ELSE
                        RETURN 0;
                END IF;
        END;
END;
```

Con ese ejemplo, el usuario con login 'luitom64' se considera mayor que el usuario 'caragu62', ya que '64' es mayor que '62'.

El método debe retornar un número negativo, cero, o un número positivo que significará que el objeto que utiliza el método (SELF) es menor, igual, o mayor que el objeto pasado por parámetro.

Debes tener en cuenta que puedes declarar **un método MAP o un método ORDER, pero no los dos.**

Cuando se vaya a ordenar o mezclar un alto número de objetos, es preferible usar un método MAP, ya que en esos casos un método ORDER es menos eficiente.

En el siguiente enlace puedes encontrar el código completo de un ejemplo de declaración y uso de un método ORDER para establecer el orden entre objetos del tipo Usuario. Como puedes comprobar, las comparaciones se realizan un función de los dos últimos dígitos del atributo login.

0:00

Resumen textual alternativo

Autoevaluación

¿Los métodos MAP sólo sirven para evaluar expresiones lógicas que resultan valores booleanos?

- Verdadero.
- Falso.

6.- Tipos de datos colección.

Caso práctico

Ana ha estudiado los vectores y matrices que se usan en varios lenguajes de programación. Le pregunta a **Juan** si con las bases de datos objeto-relacionales se puede utilizar algo parecido para almacenar los objetos instanciados. Él le dice que para eso existen las colecciones.

Para que puedas almacenar en memoria un conjunto de datos de un determinado tipo, la base de datos de Oracle te ofrece los tipos de datos **colección**.

Una **colección** es un conjunto de elementos del mismo tipo. Puede compararse con los **vectores y matrices** que se utilizan en muchos lenguajes de programación. En este caso, las colecciones sólo pueden tener **una dimensión** y los elementos se indexan mediante un valor de tipo numérico o cadena de caracteres.

La base de datos de Oracle proporciona los tipos VARRAY y NESTED TABLE (tabla anidada) como tipos de datos colección.

- Un VARRAY es una colección de elementos a la que se le establece una dimensión máxima que debe indicarse al declararla. Al tener una longitud fija, la eliminación de elementos no ahorra espacio en la memoria del ordenador.
- Una NESTED TABLE (**tabla anidada**) puede almacenar cualquier número de elementos. Tienen, por tanto, un tamaño dinámico, y no tienen que existir forzosamente valores para todas las posiciones de la colección.
- Una variación de las tablas anidadas son los **arrays asociativos**, que utilizan valores arbitrarios para sus índices. En este caso, los índices no tienen que ser necesariamente consecutivos.

Cuando necesites almacenar un número fijo de elementos, o hacer un recorrido entre los elementos de forma ordenada, o si necesitas obtener y manipular toda la colección como un valor, deberías utilizar el tipo VARRAY.

En cambio, si necesitas ejecutar consultas sobre una colección de manera eficiente, o manipular un número arbitrario de elementos, o bien realizar operaciones de inserción, actualización o borrado de forma masiva, deberías usar una NESTED TABLE.

Las colecciones pueden ser declaradas como una instrucción SQL o en el bloque de declaraciones de un programa PL/SQL. El tipo de dato de los elementos que puede contener una colección declarada en PL/SQL es cualquier tipo de dato PL/SQL, excepto REF CURSOR. Los elementos de las colecciones declaradas en SQL, además **no pueden ser de los tipos**: BINARY_INTEGER, PLS_INTEGER, BOOLEAN, LONG, LONG RAW, NATURAL, NATURALN, POSITIVE, POSITIVEN, REF CURSOR, SIGNTYPE, STRING.

Cualquier tipo de objetos declarado previamente puede ser utilizado como tipo de elemento para una colección.

Una tabla de la base de datos puede contener **columnas que sean colecciones**. Sobre una tabla que contiene colecciones se podrán realizar operaciones de consulta y manipulación de datos de la misma manera que se realiza con tablas con los tipos de datos habituales.

Para saber más

En este documento puedes encontrar más información sobre los tipos de datos colección entre otros conceptos de las bases de datos objetos-relacionales:

Soporte objeto-relacional en Oracle 8. (0.36 MB)

Autoevaluación

¿Qué tipo de colección tiene un tamaño fijo?

- VARRAY.
- NESTED TABLE.
- Array Asociativo.

8.1.- Declaración y uso de colecciones.

Las **colecciones** de arrays tipos nos ofrecen tres tipos de formas diferentes:

Vectores multidimensionales: Son los arrays que ya conocemos, donde cada elemento del array es un vector y todos los elementos que pertenecen a ese vector tienen el mismo tipo de dato. Son los clásicos arrays que hemos visto hasta ahora.

Vectores irregulares: Son los vectores de vectores de toda la vida, pero en este caso, cada vector puede tener un tamaño diferente. Es decir, cada elemento del array de vectores (cada vector) puede tener tamaños distintos.

Tablas hash: Son un tipo de dato que no es un array, pero que se utiliza para guardar parejas de datos. Cada pareja de datos está formada por una clave y un valor. Una clave es un dato único que identifica a cada pareja, y un valor es el dato que queremos guardar. Por ejemplo, podríamos tener una tabla hash que guarde parejas de nombres de personas y sus números de teléfono. En este caso, el nombre de la persona sería la clave y el número de teléfono sería el valor.

Para **declarar una de las colecciones** de la relación con **DimTipo**, se debe utilizar el nombre de la colección seguido de los parámetros, del mismo tipo que el elemento de la colección. Vamos con **VECTOR** como con **VECTOR 2 dim**, el primer elemento hace que el índice 1 sea el primero, para acceder a un elemento de una de estas dos formas(1) es necesario utilizar paréntesis.

En el siguiente ejemplo, se modifica el valor de la posición (1,1)

En el siguiente ejemplo puedes comprobar cómo puedes diferenciar las colecciones para almacenar sus datos en una tabla ilustrada de datos, así como la declaración con variables de vectores y posteriormente de los datos de la colección que se almacenarán en tabla.

```
TYPE cuadro_fijo IS VARRAY (cuadro_max) OF fijo_cuadro;
TYPE cuadro_fijo IS TABLE OF fijo_cuadro;
TYPE cuadro_fijo IS TABLE OF fijo_cuadro INDEX BY key_cuadro;
```

```
                    DECLARE TYPE cuadro_fijo IS ...
```

```
declare
    TYPE tcolores IS TABLE OF VARCHAR2(20);
    colores tcolores;
begin
    colores(1) := 'Blanco';
end;
```

```
declare
    TYPE tcolores IS TABLE OF VARCHAR2(20);
    colores tcolores := tcolores('Rojo', 'Naranja', 'Amarillo', 'Verde', 'Azul');
```

```
                    dbms_output.put_line(colores(1));
```

```
                    colores(1) := 'Gris';
```

```
DECLARE TYPE tcolores2 IS TABLE OF VARCHAR2(20);
    colores tcolores := tcolores('Rojo', 'Naranja', 'Amarillo', 'Verde', 'Azul');
    CURSOR tdatos colores2 IS SELECT * FROM colores_tdat;
BEGIN
    FOR i IN colores.FIRST..colores.LAST LOOP
        INSERT INTO tdatos2 VALUES('Datos', i, colores(i), 'Rojo', 'Amarillo', 'Verde');
    END LOOP;
    UPDATE tdatos2 SET col1 = 'Datos2' WHERE col2 = 1;
    UPDATE tdatos2 SET col2=colores(2) WHERE col2 = 2;
    UPDATE tdatos2 SET col3=colores(3) WHERE col3 = 'Amarillo';
    FOR i IN tdatos2.FIRST..tdatos2.LAST LOOP
        SELECT col1||col2||col3||col4||col5||col6 INTO datos2 FROM tdatos2 WHERE col2 = i;
        DBMS_OUTPUT.PUT_LINE(datos2);
    END LOOP;
END;
```

7.- Tablas de objetos.

Caso práctico

Ana y Mario ya conocen bien el almacenar objetos en memoria gracias a los ejemplos de objetos, cuando almacenamos la cantidad de un artículo en la tienda, **stock** que almacenaba que puede cambiarse tras hacer la compra, pero ahora se encuentran que no tienen un tipo de dato definido en la base de datos.

Piensan de inicio crear una por cada artículo en el inventario, pero son un valor agrupado que necesitan **almacenar los objetos en tablas** de igual manera que se hacía en datos de las bases de datos.

Un ejemplo de tabla, donde se pueden ver almacenados diferentes tipos de datos, como texto, fecha, y otros, pero también almacenar tipos de datos más complejos, como objetos de tipo.

Por tanto de igual manera que **existe formula de almacenamiento para un determinado tipo de dato objeto**, donde se copiaba ciertos atributos de una clase en objeto de la siguiente manera:

```
clase01 Tabla1 (nuevaTabla(), 10, TipoObj(pos));
```

Para la función **fijClase** el código a pasar debería ser un constructor con objetos como **fijClase**. Por ejemplo, para crear la tabla clase01obj, sería almacenar objetos del tipo clase01

```
clase01 Tabla1 (nuevaTabla(), 10, clase01);
```

Habría también un método para el número de filas, **fijTabla** obtener **la posición almacenada en memoria de determinado tipo de objeto**. Por tanto, desde el inventario el ejemplo, los objetos serían **filas** en el inventario, no podrían volver a definirlo.

Para saber el valor **tipo de objeto** almacenado en una tabla, se utiliza **getClase**, que devuelve el tipo de objeto que se ha almacenado.

En este caso, debe de ser declarado, **setClase** recibe como parámetro **almacenar** objetos del tipo clase01 en una tabla de la memoria de datos, **getClase01** devuelve un valor **getClase01obj** identifica a cada elemento de la tabla, obteniendo de manera independiente objetos guarda. Tipo: **getClase** en variables, por lo que **almacenar** de operación. Son objetos, y la información que contienen, desaparecerá. Si esta objeto se almacenará, en tablas de desaparecerá hasta que **almacenar** una variable de la que se almacenará.

Cuando la instancia en objeto cierra la aplicación en un variable, **delete** objeto cuando **clase01obj** hasta de la tabla la memoria de datos. Sin embargo, al fijo de objeto puede interpretar **delete** de cualquier tabla, ¿puede ser usado para crear objetos en cualquier tabla?

Las tablas, a la vez contienen filas de objetos, similar al contenido de **tablas de objetos**.

En la siguiente imagen vemos nuestra información dentro tablas que luego las filas de objetos de tipo cliente. Hacerlo a la vez definirlo del tipo de objeto se visualiza como la fuente las columnas de la tabla.

Para saber más

En este documento podrás encontrar más detalle sobre las tablas de objetos, con algunos ejemplos de creación de tablas de objetos

Manual de Tablas de Productos en Objetos (1 mes).

7.1.- Tablas con columnas tipo objeto.

Puedes usar cualquier tipo de objeto, ya sea por declararlo previamente, para utilizarlo como un **tipo de dato de una columna/atributo de una tabla** de la misma BD. Aquí, una vez declarado el objeto, podemos utilizar cualquiera de las características vistas para insertar un objeto, seleccionar sus atributos y actualizar sus datos.

Para hacer una tabla/clase de un objeto solo hay que crear una tabla cuyas columnas tengan nombres de los atributos del objeto y que sean del tipo de dato definido previamente al tipo de objeto.

Por ejemplo, podemos crear una tabla que se llame **gc_alumnos** con las columnas de objetos del tipo t_alumno que hemos utilizado anteriormente.

```
CREATE TABLE gc_alumnos OF t_alumno;
```

Como puedes comprobar en la siguiente imagen, la tabla del ejemplo almacenará los datos según t_alumno, la diferencia fundamental de objeto que hay con el ejemplo de arriba es que ahora los atributos del tipo de objeto t_alumno no se muestran como un hueco/s sobre el hueco/ valores almacenados de la tabla, sino que forman parte de una misma columna.

> **Autoevaluación**
>
> Una tabla con columnas de tipo objeto, ¿en cuántas de sus columnas puede almacenar un objeto?
>
> - Una.
> - Ninguna.

7.2.- Uso de la sentencia Select.

De manera similar a las consultas que has realizado sobre tablas sin tipos de objetos, **puedes utilizar la sentencia SELECT** para obtener datos de las filas almacenadas en tablas de objetos o tablas con columnas de tipos de objetos.

El uso más sencillo sería para mostrar todas las filas contenidas en la tabla:

```
SELECT * FROM NombreTabla;
```

Como puedes apreciar en la imagen, la tabla que forme parte de la consulta puede ser una tabla de objetos (como la tabla UsuariosObj), o una tabla que contiene columnas de tipos de objetos (como la tabla Gente).

En las sentencias SELECT que utilices con objetos, puedes incluir cualquiera de las **cláusulas y funciones de agrupamiento** que has aprendido para la sentencia SELECT que has usado anteriormente con las tablas que contienen columnas de tipos básicos. Por ejemplo, puedes utilizar: SUM, MAX, WHERE, ORDER, JOIN, etc.

Es habitual utilizar **alias** para hacer referencia al nombre de la tabla. Observa, por ejemplo, la siguiente consulta, en la que se desea obtener el nombre y los apellidos de los usuarios que tienen algo de crédito:

```
SELECT u.nombre, u.apellidos FROM UsuariosObj u WHERE u.credito > 0
```

Si se trata de una **tabla con columnas de tipo objeto**, el acceso a los atributos del objeto se debe realizar indicando previamente el nombre asignado a la columna que contiene los objetos:

```
SELECT g.unUsuario.nombre, g.unUsuario.apellidos FROM Gente g;
```

Para saber más

En este documento puedes encontrar información general sobre las bases de datos objeto-relacionales, con algunos ejemplos de uso de la sentencia SELECT con tablas de objetos:

Tipos de Objeto en PL/SQL. (1.28 MB)

7.3.- Inserción de objetos.

Evidentemente, no te servirá de nada una tabla que pueda contener objetos sino conocemos la manera de insertar objetos en la tabla.

La manera que tienes para ello **es la misma** que has utilizado para introducir datos de cualquier tipo habitual en las tablas de la base de datos: usando la sentencia INSERT de SQL.

En las tablas habituales, cuando querías añadir una fila a una tabla que tenía un campo VARCHAR le suministrabas a la sentencia INSERT un dato de ese tipo. Pues si la tabla es de un determinado tipo de objetos, o si posee un campo de un determinado tipo de objeto, tendrás que suministrar a la sentencia INSERT un objeto instanciado de su tipo de objeto correspondiente.

Por tanto, si queremos insertar un Usuario en la tabla Gente que hemos creado en el apartado anterior, previamente debemos crear el objeto o los objetos que se deseen insertar. A continuación podremos utilizarlos dentro de la sentencia INSERT como si fueran valores simplemente.

```
DECLARE
        u1 Usuario;
        u2 Usuario;
BEGIN
        u1 := NEW Usuario('luitom64', 'LUIS', 'TOMAS BRUNA', '24/10/2007', 50);
        u2 := NEW Usuario('caragu72', 'CARLOS', 'AGUDO SEGURA', '06/07/2007', 100);
        INSERT INTO UsuariosObj VALUES (u1);
        INSERT INTO UsuariosObj VALUES (u2);
END;
```

De una manera más directa, puedes crear el objeto dentro de la sentencia INSERT directamente, sin necesidad de guardar el objeto previamente en una variable:

```
INSERT INTO UsuariosObj VALUES (Usuario('luitom64', 'LUIS', 'TOMAS BRUNA', '24/10/2007', 50));
```

Podrás comprobar los resultados haciendo una consulta SELECT sobre la tabla de la manera habitual:

```
SELECT * FROM UsuariosObj;
```

De manera similar puedes realizar la **inserción de filas en tablas con columnas de tipo objeto**. Por ejemplo, para la tabla Gente que posee entre sus columnas, una de tipo objeto Usuario, podríamos usar el formato de instanciar el objeto directamente en la sentencia INSERT, o bien, indicar una variable que almacena un objeto que se ha instanciado anteriormente:

```
INSERT INTO Gente VALUES ('22900970P', Usuario('luitom64', 'LUIS', 'TOMAS BRUNA', '24/10/2007', 50), 54);

INSERT INTO Gente VALUES ('62603088D', u2, 21);
```

7.4.- Modificación de objetos

En bases modifica un objeto almacenado en una tabla lo hará tanto para añadir **nuevas ocurrencias fruto** de que alguien haya añadido una tupla, como para modificar algunos de sus campos. ¿Recuerdas la sentencia UPDATE? Pues también la vamos a usar en bases para modificar los objetos de la tabla, es igual, tenemos que acceder al objeto que queremos y ya está.

Hay una pequeña diferencia con el caso de la sentencia de la creación de nuevos objetos (INSERT), en ese caso, **debíamos** acceder a la clase para crear un nuevo objeto, y luego ya podíamos acceder a alguno de ellos. Ahora no hace falta.

En una tabla de una determinada clase, se hace referencia a los objetos que están dentro del contexto asignado a la tabla. Recordad existen dos formas siguientes:

```
UPDATE <tabla_objetos>
    SET <atributo>.<atributo_de_clase>=<nuevo_valor>
    WHERE <nombre_objeto_del_que_queremos_modificar>.<atributo_de_clase>=<nuevo_valor>;
```

Podríamos decir que agregado simplemente entendiendo como ocurrencia que deseamos modificar los datos de un determinado usuario. Por ejemplo, modificaríamos el nombre del usuario identificado por el login "nombre1" asignándole el valor 'X':

```
UPDATE <nombre_tabla>
    SET <nombre_tabla>.<campo> =x
    WHERE <nombre_tabla>.<login> = "nombre1" ;
```

De esta manera estaríamos accediendo a la tabla con el **alias**

```
UPDATE <nombre_tabla> x
    SET x.<campo> = x
    WHERE x.<login> = "nombre1" ;
```

Pero no sólo podemos modificar el nombre de una determinada atributo de objeto. Podemos **modificar un objeto por entero** como queda con el siguiente ejemplo, en el que vamos modificando para usuario con login 'mortag78' por otro usuario nuevo:

```
UPDATE <nombre_tabla> x SET x = <nuevo>('juana2018', 'Juan', 'fernández valdivieso', '65.432.765-K', 32, 'a2000', x.<login>, 'mortag78') ;
```

En una tabla de una determinada clase de tipo objeto, se debe hacer referencia al nombre tanto atributos que se deseen los objetos:

```
UPDATE <tabla_objetos>
    SET <tabla_objetos.nombre_atributo_del_objeto>=<nuevo_valor>
    WHERE <tabla_objetos.nombre_atributo_del_objeto>=<nuevo_valor>;
```

A continuación podéis ver el ejemplo de actualización de datos de la tabla de la tabla que se habría creado con una función de tipo tu objeto basada. Recuerda que de, seleccionar de que se almacenan los objetos de tipo Usuario: esta tabla asignado un nombre al usuario:

```
UPDATE <tabla_x>
    SET x.<atributo>=x.<nuevo_valor>
    WHERE x.<atributo>.<login> = "nombre1" ;
```

Ahora, puedes modificar todos un objeto por ahí, mantenerse en este tan todos de la fila en modificar, como el ejemplo agregado, mientras tanto **UPDATE** podríamos agregar no se sabemos, aún no sabíamos usuario y no otro.

```
UPDATE <tabla_x>
    SET x.<atributo>=x.<nuevo>('juana2018', 'Juan', 'fernández valdivieso', '65.432.765-K', 32
    WHERE x.<atributo>.<login> = "mortag78" ;
```

7.5.- Borrado de objetos.

En el capítulo 1 ya vimos cómo crear un fichero de tipo .mxe (ejecutable) para poder ejecutar directamente el objeto sin necesidad de tener el entorno de desarrollo instalado en el ordenador. Vamos a ver ahora cómo hacerlo. Es importante tener en cuenta que para poder utilizar un fichero .mxe en otro ordenador, éste debe tener instalado el mismo entorno que se ha utilizado para crearlo. Es decir, si hemos creado el fichero .mxe con la versión 2010 de Visual FoxPro, el ordenador donde queramos ejecutarlo también deberá tener instalada la misma versión.

En este caso, el objeto BORRAR, además de tener las propiedades y métodos que hemos visto hasta ahora, también tendrá un método nuevo llamado Eliminar.

Este método se encargará de eliminar el objeto que le indiquemos en la propiedad **cObjeto**. Para ello, primero comprobará que el objeto existe y, en caso afirmativo, lo eliminará.

Para saber si el objeto existe o no, utilizaremos la función **PEMSTATUS()**. Esta función nos devuelve .T. si el objeto que le indicamos existe y .F. en caso contrario. Recibe tres parámetros: el primero es el nombre del objeto, el segundo es el nombre de la propiedad o método que queremos comprobar y el tercero es un número que indica si estamos comprobando una propiedad o un método. Si el objeto que le indicamos no existe, la función devuelve .F. sin importar el valor que tengan los otros dos parámetros.

Si el objeto existe, entonces comprobaremos si tiene una propiedad o método llamado Eliminar. Si no existe, también devolverá .F.. Si existe, entonces ejecutaremos el método Eliminar() y devolveremos .T.. Si el método Eliminar() devuelve .F., entonces también devolveremos .F.. Si el método Eliminar() devuelve .T., entonces también devolveremos .T..

Veamos el código del método Eliminar():

```
DO CASE
	CASE EMPTY(This.cObjeto)
		RETURN .F.
	CASE PEMSTATUS(This.cObjeto, 'Eliminar', 5) = 0
		RETURN .F.
	OTHERWISE
		RETURN This.cObjeto.Eliminar()
ENDCASE
```

Como podemos ver, primero comprobamos si la propiedad cObjeto está vacía. Si lo está, devolvemos .F.. Si no lo está, comprobamos si el objeto que nos indican en la propiedad cObjeto tiene un método llamado Eliminar. Si no lo tiene, devolvemos .F.. Si lo tiene, ejecutamos el método Eliminar() del objeto que nos indican en la propiedad cObjeto y devolvemos el valor que devuelva el método Eliminar() del objeto que nos indican en la propiedad cObjeto.

Veamos el código del método Eliminar() de la clase BORRAR:

```
LOCAL lnSeleccionado, lnContador, lnTotal, lcObjeto
lnSeleccionado = This.oLista.ListIndex
IF lnSeleccionado = 0
	RETURN .F.
ENDIF
lnTotal = This.oLista.ListCount
FOR lnContador = 1 TO lnTotal
	IF This.oLista.Selected(lnContador)
		lcObjeto = This.oLista.List(lnContador)
		IF This.oObjetos.Eliminar(lcObjeto)
			This.oLista.RemoveItem(lnContador)
		ENDIF
	ENDIF
ENDFOR
This.oLista.ListIndex = lnSeleccionado
RETURN .T.
```

Como podemos ver, primero comprobamos si hay algún elemento seleccionado en el cuadro de lista oLista. Si no lo hay, devolvemos .F.. Si lo hay, recorremos todos los elementos del cuadro de lista oLista y comprobamos si cada uno de ellos está seleccionado o no. Si está seleccionado, ejecutamos el método Eliminar() del objeto que nos indican en la propiedad cObjeto y, si el método Eliminar() devuelve .T., entonces eliminamos el elemento del cuadro de lista oLista. Una vez hemos recorrido todos los elementos del cuadro de lista oLista, volvemos a dejar seleccionado el mismo elemento que estaba seleccionado antes de empezar a recorrer todos los elementos del cuadro de lista oLista y devolvemos .T..

Ejecutamos el proyecto y probamos a eliminar algún objeto. Veremos que, efectivamente, se eliminan todos los objetos que hemos seleccionado en el cuadro de lista oLista.

Ahora vamos a ver cómo hacer que, cuando ejecutemos el proyecto, se muestre la pantalla de inicio. Para ello, vamos a crear un nuevo formulario llamado Inicio.scx. Le vamos a añadir un control de tipo Image llamado imgImagen y le vamos a cargar la imagen Inicio.jpg que hay en la carpeta Imagenes. Ajustamos el tamaño del formulario y de la imagen para que la imagen ocupe todo el formulario. Y, por último, añadimos un control de tipo Timer llamado tmrTemporizador. Le vamos a dar un valor de 3000 a su propiedad Interval. Y, en el método Timer de tmrTemporizador, vamos a escribir el siguiente código:

```
This.Release()
```

Ejecutamos el proyecto y comprobamos que, efectivamente, se muestra la pantalla de inicio durante 3 segundos y, a continuación, se muestra la pantalla principal del programa.

Para terminar, el último paso que vamos a dar es el de crear un fichero .mxe para poder ejecutar el programa directamente sin tener que abrir el entorno de desarrollo. Para ello, vamos a crear un nuevo proyecto llamado Principal.pjx. Le añadimos todos los ficheros que hemos creado hasta ahora, excepto el fichero Inicio.scx. Y, por último, le indicamos que queremos crear un fichero .mxe llamado Principal.mxe y que queremos que el fichero .mxe se ejecute en modo Pantalla. Pulsamos el botón Aceptar y ya tenemos creado el fichero .mxe.

Ejecutamos el fichero Principal.mxe y comprobamos que, efectivamente, se muestra la pantalla de inicio durante 3 segundos y, a continuación, se muestra la pantalla principal del programa.

7.4.- Consultas con la función VALUE.

Cuando hacemos referencia a **base relacional a un objeto de Angular lo ingresado sea cual fuera**, puedes utilizar la función nativa para usar un servicio de la Web de Objetos o La web, **dentro de una arrow function** R: Puedes ver a continuación un ejemplo de una de estas función, para hacer referencia a una tabla (Favorito) de nuestro tipo de objeto:

```
SELECT VALUE favorito
FROM favorito
WHERE favorito.usuario = "usuario1"
```

Esta función VALUE nos da la posibilidad utilizarla para hacer comparaciones de igualdad entre objetos, por ejemplo, si deseamos obtener datos de un usuario cuyo es almacenado en una tabla (favorito) y (usuario999):

```
SELECT u.login, u.edad, favorito.id_usuario
FROM usuario u, VALUE favorito
WHERE u.login = "usuario999"
```

Observa la diferencia en el uso cuando se hace la comparación con una columna de tipo de objeto. En este caso la referencia que se hace a la columna (u.id_usuario) permite obtener directamente el objeto, sin necesidad de utilizar la función VALUE

```
SELECT g.id, g.titulo, g.fecha_lanzamiento, g.precio
FROM juego g
WHERE g.id_usuario IN (SELECT VALUE u.id
                       FROM usuario u
                       WHERE u.login = "usuario1");
```

Cuando la **función VALUE** se **utiliza para un atributo u objeto referido** en las consultas usando la función nativa VALUE, como por ejemplo cuando un objeto, o un atributo (entidad fuerte) con la clave de nuestra tabla, tenemos hechas (que hace valer entre tablas/relaciones) la comparación de objetos. Por ejemplo, podemos recibir a una entidad/tipo, tercer parte de asignaciones, así, En el siguiente ejemplo, se realiza una consulta de la tabla/clase(BD), para recibir un determinado objeto del tipo Usuario. El objeto resultante de la consulta es guardado en la tabla U. Esa consulta se utiliza, para recibir los (posibles) archivos del usuario, y para de esa lógica sea usable, para visualizarlos en distintos lugares de pantalla.

```
CREATE
    or replace
    or replace
PROCEDURE
    SELECT VALUE u FROM usuario u WHERE u.login = "usuario1";
    dbms_output.put_line(usuario);
    SELECT VALUE u FROM usuario u WHERE u.login = "usuario2";
    dbms_output.put_line(usuario);
END;
```

Para saber más

En este documento te podrás encontrar información general sobre las bases de datos Objeto-relacionales, navegación relacional y ejemplos de la función VALUE

Manual de bases objeto-relacionales (enlace)

7.7.- Referencias a objetos.

El paso de objetos a un método resulta ineficiente cuando se trata de objeto de gran tamaño, por lo que es más conveniente **pasar un puntero a dicho objeto**, lo que permite que el método que lo recibe pueda hacer referencia a dicho objeto sin que sea necesario que se pase por completo. Ese puntero es lo que se conoce en Oracle como una **referencia (REF)**.

Al compartir un objeto mediante su referencia, **los datos no son duplicados**, por lo que cuando se hace cualquier cambio en los atributos del objeto, se producen en un único lugar.

Cada objeto almacenado en una tabla tiene un **identificador de objeto** que identifica de forma única al objeto guardado en una determinada fila y sirve como una referencia a dicho objeto.

Las referencias se crean utilizando el modificador REF delante del tipo de objeto, y se puede usar con variables, parámetros, campos, atributos, e incluso como variables de entrada o salida para sentencias de manipulación de datos en SQL.

```
CREATE OR REPLACE TYPE Partida AS OBJECT (
        codigo INTEGER,
        nombre VARCHAR2(20),
        usuarioCreador REF Usuario
);
/

DECLARE
        u_ref REF Usuario;
        p1 Partida;
BEGIN
        SELECT REF(u) INTO u_ref FROM UsuariosObj u WHERE u.login = 'luitom64';
        p1 := NEW Partida(1, 'partida1', u_ref);
END;
/
```

Hay que tener en cuenta que sólo se pueden usar **referencias a tipos de objetos que han sido declarados previamente**. Siguiendo el ejemplo anterior, no se podría declarar el tipo Partida antes que el tipo Usuario, ya que dentro del tipo Partida se utiliza una referencia al tipo Usuario. Por tanto, primero debe estar declarado el tipo Usuario y luego el tipo Partida.

El problema surge cuando tengamos dos tipos que utilizan referencias mutuas. Es decir, un atributo del primer tipo hace referencia a un objeto del segundo tipo, y viceversa. Esto se puede solucionar haciendo una **declaración de tipo anticipada**. Se realiza indicando únicamente el nombre del tipo de objeto que se detallará más adelante:

```
CREATE OR REPLACE TYPE tipo2;
/
CREATE OR REPLACE TYPE tipo1 AS OBJECT (
        tipo2_ref REF tipo2
        /*Declaración del resto de atributos del tipo1*/
);
/
CREATE OR REPLACE TYPE tipo2 AS OBJECT (
        tipo1_ref REF tipo1
        /*Declaración del resto de atributos del tipo2*/
);
/
```

7.5.- Navegación a través de referencias

Debes tener en cuenta que **el puntero no sabe directamente a las atribuciones de un objeto referenciado por una referencia**. Para ello, puedes utilizar la flecha (->) como alias:

Esta notación **(*referencia).atributo** es un **alias** de **referencia->atributo**

Debes conocer un concepto importante que diferencia de las siguientes variables declaradas:

```
int MAX_TAMAÑO;
int *pMAX_TAMAÑO;
```

Si **pMAX_TAMAÑO** debe almacenar la dirección de MAX_TAMAÑO, para obtener el **valor** de MAX_TAMAÑO usamos el operador de la dirección de MAX_TAMAÑO, es decir, **&MAX_TAMAÑO**.

Ahora bien, con el fin de **poder gestionar de una manera eficiente** MAX_TAMAÑO, por lo que hay que definir una función llamada **MAX**. Dicha función, que debe **recibir un puntero** a MAX_TAMAÑO, para poder acceder a su valor, por lo que la información se trabaja siempre **directamente** en el objeto apuntado almacenado en una **referencia** (MAX). Por tanto, no debería ser necesario indicar de cuántos caracteres se trata MAX. Finalmente hacer referencia a cualquier fallo en la función, ¿y ha sido **ERROR** o bien **ALGORITMO**? Utilizaremos **¿qué es el manual error**, es decir, cuando se produzca un error.

Y, como ves, todos los errores, ¿ha sido **ERROR** o bien **ALGORITMO**? Utilizaremos **¿qué es el manual error**, es decir, cuando se produzca un error.

```
MAX(&MAX_TAMAÑO, pMAX_TAMAÑO);
MAX(pMAX_TAMAÑO, &MAX_TAMAÑO);
```

Por tanto, para obtener un objeto referenciado por una variable MAX, debes **utilizar una consulta sobre cualquier tabla**, siempre teniendo en cuenta que la tabla a la que se accede a través de la tabla a la que se accede a través de un objeto referenciado. Más sobre la creación de una consulta en Access, dirígete una vez hayas finalizado. La tabla **relación en utilizar una tabla** MAX, es decir, la que se accede tabla a la tabla MAX, **¿ha sido MAX o bien ALGORITMO?** Utilizaremos **¿qué es el manual error**, es decir, cuando se produzca un error.

Para saber más

En este documento, puedes encontrar información general sobre bucles for anidados, así como ejemplos y comparativa ejemplos de tablas de objetos con uso de referencias.

Anexo - Licencias de recursos.

Licencias de recursos utilizados en la Unidad de Trabajo.

Recurso (1)	Datos del recurso (1)	Recurso (2)	Datos del recurso (2)
	Autor/a: Ministerio de Educación Licencia: Uso Educativo. Uso personal. Procedencia: Material de Lengua Castellana. En http://cnice.mecd.es/media/image/publicidad/banner_cervantes.jpg		Autor/a: Ministerio de Educación Licencia: Uso Educativo. Uso personal. Procedencia: Material de Lengua Castellana. En http://cnice.mecd.es/media/image/publicidad/banner_cervantes.jpg
	Autor/a: Mark Hayter Licencia: Free Documentation License Procedencia: http://commons.wikimedia.org/wiki/File:Magnifier.svg		Autor/a: The people find the Thing Project Licencia: Free Documentation License Procedencia: http://commons.wikimedia.org/wiki/File:Edit_options.svg
	Autor/a: Gottfried Hein Licencia: GNU/GPL Procedencia: http://commons.wikimedia.org/wiki/File:Applications-system.svg		Autor/a: The people find the Thing Project Licencia: Free Documentation License Procedencia: http://commons.wikimedia.org/wiki/File:Kontact.png
	Autor/a: Ministerio de Educación Licencia: Uso Educativo. Uso personal Procedencia: Material de Lengua Castellana. Imagenes de terminos y palabra 1. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 2. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg		Autor/a: Ministerio de Educación Licencia: Uso Educativo. Uso personal Procedencia: Material de Lengua Castellana. Imagenes de terminos y palabra 1. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 2. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 3. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 4. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg
	Autor/a: Tuxedo Cowboy and Tiberiandus Licencia: GNU/GPL Procedencia: http://commons.wikimedia.org/wiki/File:Crystal_Clear_Device_Monitor.svg		Autor/a: Ministerio de Educación Licencia: Uso Educativo. Uso personal Procedencia: Material de Lengua Castellana. Imagenes de terminos y palabra 1. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 2. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 3. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 4. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg
	Autor/a: Oliver1 Licencia: GNU/GPL Procedencia: 7000Comm01 Pizza. Tarjeta de pantalla de la marca comercial Alto de Sierra Nakabara		Autor/a: Ministerio de Educación Licencia: Uso Educativo. Uso personal Procedencia: Material de Lengua Castellana. Imagenes de terminos y palabra 1. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 2. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 3. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 4. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg
	Autor/a: Oliver1 Licencia: GNU/GPL Procedencia: 7000Comm01 Pizza. Tarjeta de pantalla de la marca comercial Alto de Sierra Nakabara		Autor/a: Oliver1 Licencia: GNU/GPL Procedencia: 7000Comm01 Pizza. Tarjeta de pantalla de la marca comercial Alto de Sierra Nakabara
	Autor/a: Ministerio de Educación Licencia: Uso Educativo. Uso personal Procedencia: Material de Lengua Castellana. Imagenes de terminos y palabra 1. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 2. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 3. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 4. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg		Autor/a: Ministerio de Educación Licencia: Uso Educativo. Uso personal Procedencia: Material de Lengua Castellana. Imagenes de terminos y palabra 1. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 2. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 3. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 4. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg
	Autor/a: Ministerio de Educación Licencia: Uso Educativo. Uso personal Procedencia: Material de Lengua Castellana. Imagenes de terminos y palabra 1. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 2. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 3. http://commons.wikimedia.org/wiki/File:Archivo-comun-pruebas.svg 4. http://commons.wikimedia.org/wiki/File:Archivo-comun.svg		